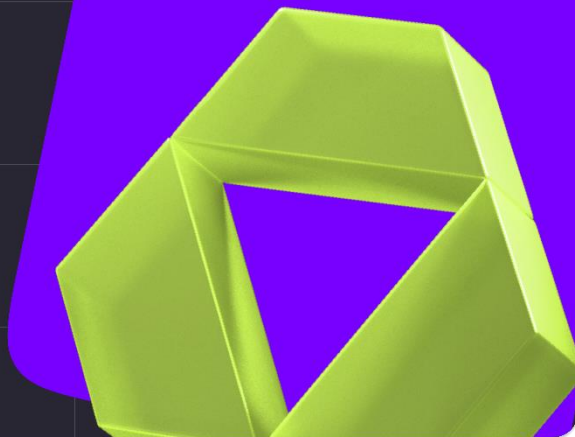


Преридер

SQL для начинающих
Занятие 5



Описание

Цель занятия — освоить работу с агрегатными функциями, группировкой данных и оконными функциями для анализа информации в базах данных.

Что вспомнить до занятия

● Базовый синтаксис SELECT, FROM, WHERE

Вспомним, как выглядит базовый синтаксис выбора данных.

```
SELECT столбцы  
FROM таблица  
WHERE условие;
```

Посмотрите пример запроса, который выводит все строки и все столбцы из таблицы **clients**.

```
SELECT *  
FROM clients;
```

А вот так можно сделать фильтрация по условию.

```
SELECT first_name, last_name  
FROM clients  
WHERE first_name ILIKE 'A%';
```

Этот запрос позволит отобразить фамилии и имена всех клиентов, чьи имена начинаются на А.

Что изучить до занятия

● Оконные функции

Оконные функции — это специальные функции в SQL, которые:

Выполняют вычисления по группе строк, связанных с текущей строкой;

Не «сворачивают» строки, как агрегатные функции (GROUP BY);

Возвращают результат для каждой строки исходной таблицы.

Вот так выглядит общий синтаксис:

```
<оконная_функция>() OVER (  
    [PARTITION BY столбец1, ...] -- разбивает данные на группы («окна»)  
    [ORDER BY столбец2, ...] -- задаёт порядок внутри окна  
    [ROWS/RANGE BETWEEN ...] -- определяет рамки окна (необязательно)  
)
```

Т.е. оконные функции позволяют выполнять вычисления по «окну» строк, связанному с текущей строкой, не сворачивая исходный набор данных. Каждая строка остаётся в результате, но к ней добавляется вычисленное значение.

Пример

Посмотрим, как можно сравнить текущее значение с предыдущим и следующим платежом за день бронирования.

```
SELECT  
    booking_id,  
    payment_date,  
    amount_paid,  
    LAG(amount_paid) OVER (PARTITION BY payment_date ORDER BY booking_id) AS  
    prev_amount,  
    LEAD(amount_paid) OVER (PARTITION BY payment_date ORDER BY booking_id) AS  
    next_amount  
FROM payments  
ORDER BY booking_id, payment_date;
```

LAG(amount) — сумма предыдущего платежа в этот день;

LEAD(amount) — сумма следующего платежа в этот день.

Оконные функции

- **ROW_NUMBER()**

Нумерует строки в окне, начиная с 1. Уникальный номер для каждой строки (даже при одинаковых значениях).

- **RANK()**

Присваивает ранг строкам. При одинаковых значениях — одинаковый ранг, но следующий ранг «пропускает» позиции (например: 1, 1, 3).

- **DENSE_RANK()**

Как RANK(), но без пропусков (например: 1, 1, 2).

- **LAG(выражение [, смещение [, значение_по_умолчанию]])**

Возвращает значение из предыдущей строки в окне (по умолчанию — на 1 строку назад).

- **LEAD(выражение [, смещение [, значение_по_умолчанию]])**

Возвращает значение из следующей строки в окне (по умолчанию — на 1 строку вперёд).

- **FIRST_VALUE(выражение)**

Возвращает значение из первой строки окна.

- **LAST_VALUE(выражение)**
Возвращает значение из последней строки окна (внимание: зависит от рамок окна!).
- **NTH_VALUE(выражение, n)**
Возвращает значение из n-й строки окна.
- **PERCENT_RANK()**
Возвращает относительный ранг строки в диапазоне от 0 до 1 (аналог $(\text{RANK}() - 1) / (\text{общее число} - 1)$).
- **CUME_DIST()**
Кумулятивное распределение: доля строк, имеющих значение $\leq$ текущего.
- **NTILE(n)**
Делит окно на n примерно равных групп и присваивает каждой строке номер группы (от 1 до n).
- Некоторые агрегатные функции тоже могут быть использованы как оконные.

Когда использовать оконные функции

- Нужно видеть исходные строки + агрегированные значения одновременно.
- Требуется сравнение с соседними записями (например, рост/падение продаж).
- Надо посчитать накопленную сумму, ранг или скользящее среднее.